

Vectors in C++ — DSA Notes

Part of my DSA placement prep journey. Previous: Arrays · **Current: Vectors** · Next: Linked Lists

Table of Contents

1. What Is a Vector?
2. Internal Working
3. Time & Space Complexity
4. Vector Operations & Functions
5. Comparison: Array vs Dynamic Array vs Vector
6. When and Where to Use Vectors
7. Question Patterns
8. Practice Problems (Categorized)
9. Worked Example
10. Common Pitfalls
11. Quick Revision Summary

1. What Is a Vector?

A **vector** (`std::vector` in C++) is a sequence container representing a **dynamic array** that can grow or shrink in size automatically when elements are inserted or deleted. It occupies a contiguous block of heap memory and manages its own memory lifecycle.

```
#include <vector>
using namespace std;

vector<int> vec;                // empty vector
vector<int> vec2(5, 10);        // size = 5, all initialized to 10
vector<int> vec3 = {1, 2, 3, 4}; // initialized with initializer list
vector<int> vec4(vec3);         // copy constructor (creates a copy)
```

Key trait: Just like arrays, vector elements sit contiguously in memory, meaning random access is $O(1)$ using index-based computation (`&vec[i] = base + i * sizeof(T)`).

2. Internal Working

2.1 Dynamic Resizing (How it Grows)

- A vector keeps track of three main parameters:

- **Size** (`vec.size()`): Number of elements currently in the vector.
- **Capacity** (`vec.capacity()`): Total number of elements the current memory block can hold before reallocation is required.
- **Base pointer**: A pointer to the heap-allocated memory block.
- When `size == capacity` and a new element is pushed:
 - The vector allocates a **new block of memory**, typically **2x** (or **1.5x** in MSVC) the current capacity.
 - It copies (or moves) all existing elements from the old block to the new block.
 - It constructs the new element in the new space.
 - It deallocates the old memory block.
 - It updates the base pointer, size, and capacity.

2.2 Amortized $O(1)$ Time Complexity

- While a single push that triggers reallocation takes $O(n)$ time, these reallocations happen very infrequently (exponentially spaced).
- Pushing N elements starting from an empty vector requires only $O(N)$ total work, which means the **average (amortized) cost per push_back** is $O(1)$.

2.3 Growth Visualization

Element Pushed	Size	Capacity	Memory Address Action
Initial	0	0	None
Push 1	1	1	Allocate 1 unit
Push 2	2	2	Double capacity to 2, copy, deallocate old
Push 3	3	4	Double capacity to 4, copy, deallocate old
Push 4	4	4	No reallocation
Push 5	5	8	Double capacity to 8, copy, deallocate old

3. Time & Space Complexity

Operation	Time Complexity	Notes
Random Access <code>vec[i]</code> or <code>vec.at(i)</code>	$O(1)$	Direct pointer offset arithmetic
Push Back <code>vec.push_back()</code>	Amortized $O(1)$	$O(n)$ when reallocation is triggered
Pop Back <code>vec.pop_back()</code>	$O(1)$	Shrinks size, capacity remains unchanged
Insert/Erase (at end)	$O(1)$	Equivalent to push/pop
Insert/Erase (middle/beginning)	$O(n)$	Requires shifting elements and/or reallocating
Reserve <code>vec.reserve(n)</code>	$O(n)$	Pre-allocates memory to avoid mid-operation copies
Shrink to fit <code>vec.shrink_to_fit()</code>	$O(n)$	Releases unused capacity (copies to a smaller block)
Clear <code>vec.clear()</code>	$O(n)$	Calls destructors for all elements; capacity stays same

Space complexity: $O(\text{capacity})$, which is at most $O(2 * n)$ for n elements. No overhead per element except for the vector header variables (24 bytes on 64-bit systems).

4. Vector Operations & Functions

```
vector<int> v = {10, 20, 30};

// 1. Modifiers
v.push_back(40);           // Add to end: {10, 20, 30, 40}
v.pop_back();              // Remove from end: {10, 20, 30}

// 2. Iterators & Insertion/Deletion at specific positions
v.insert(v.begin() + 1, 15); // Insert 15 at index 1: {10, 15, 20, 30} [O(n)]
v.erase(v.begin() + 2);     // Erase element at index 2: {10, 15, 30} [O(n)]

// 3. Size vs Capacity
cout << v.size();           // Output: 3
cout << v.capacity();       // Output: 4 (or larger)
v.reserve(100);            // Pre-allocates space for 100 elements (prevents copies)
v.shrink_to_fit();         // Releases unused capacity to match size

// 4. Element Access
cout << v[0];               // O(1) access (no bounds checking)
cout << v.at(0);            // O(1) access (throws out_of_range if index is invalid)
cout << v.front();          // First element (10)
cout << v.back();           // Last element (30)

// 5. Iteration / Traversal
for (int x : v) cout << x << " "; // Range-based for loop
for (auto it = v.begin(); it != v.end(); ++it) cout << *it << " "; // Using iterators
```

5. Comparison: Array vs Dynamic Array vs Vector

Attribute	Static Array (<code>int arr[5]</code>)	Dynamic Array (<code>new int[n]</code>)	Vector (<code>std::vector<int></code>)
Memory Location	Stack (usually)	Heap	Heap
Size Determination	Compile time	Runtime	Runtime
Resizability	Fixed, cannot change	Fixed after creation	Dynamically changes
Memory Cleanup	Automatic (goes out of scope)	Manual (<code>delete[]</code>)	Automatic (RAII)
Bound Checking	No	No	Yes (using <code>.at()</code>)
Copying	Needs manual loop/copy	Needs manual loop/copy	Direct copy supported (<code>v1 = v2</code>)

6. When and Where to Use Vectors

6.1 In DSA / Interviews

- The **default container** for linear structures when size is variable.
- Used in graphs (adjacency lists are represented as `vector<vector<int>>`).
- Backing store for dynamic grids, heaps (represented as dynamic arrays), and sorting inputs.

6.2 In Real-World Engineering

- **HTTP Payload processing:** Storing parsed items from an incoming JSON body or stream of elements when payload size varies.
- **Physics engines:** Game engines represent arrays of actors, particles, or mesh vertices in contiguous vectors to leverage CPU cache lines during rendering passes.
- **Compiler design:** Parsed AST nodes or tokens are gathered in vectors before constructing parse trees.
- **Multithreading:** Vectors are often used as thread-safe queues or task queues (protected by mutexes) where workers fetch jobs.

6.3 When NOT to Use a Vector

- Frequent insertions or deletions in the **middle** or **front** -> use `std::list` (linked list) or `std::deque`.
- Search by key needed -> use `std::unordered_map` or `std::map`.
- Strict memory-constrained systems (like tiny microcontrollers) where dynamic allocations can lead to **heap fragmentation** -> use fixed-size static arrays (`std::array` or raw arrays).

7. Question Patterns

Pattern	Typical Ask	Example Problems
Dynamic Matrix / Grid	2D vector layouts with varying dimensions	Pascal's Triangle, Search a 2D Matrix
Flattening	Converting nested vectors to a single linear vector	Flatten Nested List Iterator
Running / Sliding Stats	Dynamically updating vector buffers	Sliding Window Maximum, Find Median from Data Stream
Prefix Accumulators	Building prefix arrays on dynamic inputs	Product of Array Except Self, Subarray Sums
Invalidation Bugs	Modifying vectors while traversing	Remove Duplicates, Filter elements in-place

8. Practice Problems (Categorized)

8.1 Basic Vector Tasks

1. Write a function to check if two vectors are identical (independent of capacity).
2. Use `std::vector::reserve` to optimize pushing 1,000,000 integers and benchmark the speedup.
3. Remove all negative numbers from a vector in-place (Erase-Remove idiom).
4. Implement a 2D matrix dynamic rotation using `vector<vector<int>>`.

8.2 Matrix Patterns

1. Spiral Matrix generator (`n x n` output grid).
2. Diagonal Traversal of a 2D vector.
3. Pascal's Triangle (generate the first `numRows` rows).

8.3 Invalidation & Shifting

1. Remove Duplicates from Sorted Array (using vector resize).
2. Merge Sorted Array (merge `nums2` into `nums1` in-place).
3. Remove Element (in-place value deletion and resizing).

9. Worked Example

Problem: Pascal's Triangle

Given an integer `numRows`, generate the first `numRows` of Pascal's triangle. Each number is the sum of the two numbers directly above it.

Approach - A 2D vector `vector<vector<int>>` is perfect here since each row `i` has a different size `i + 1` (a jagged dynamic structure). - Initialize the outer vector with size `numRows`. - For each row `i`, size is `i + 1`, and the first and last elements are `1`. - For intermediate elements, `row[j] = prev_row[j - 1] + prev_row[j]`.

Code

```
vector<vector<int>>> generatePascal(int numRows) {
    vector<vector<int>>> triangle(numRows);
    for (int i = 0; i < numRows; i++) {
        triangle[i].resize(i + 1, 1); // Set size to i+1 and initialize to 1
        for (int j = 1; j < i; j++) {
            triangle[i][j] = triangle[i-1][j-1] + triangle[i-1][j];
        }
    }
    return triangle;
}
```

Complexity - Time: $O(\text{numRows}^2)$ — we compute each cell once. - Space: $O(\text{numRows}^2)$ — memory needed to store the output grid.

10. Common Pitfalls

10.1 Iterator Invalidation (CRITICAL)

If you insert or erase elements, or if a push triggers reallocation, **all iterators, references, and pointers pointing to vector elements become invalid**.

```
vector<int> v = {1, 2, 3};
for (auto it = v.begin(); it != v.end(); ++it) {
    if (*it == 2) v.push_back(4); // BUG: push_back invalidates it and v.end();
}
```

Solution: Never modify the vector size inside a standard iterator loop unless you re-assign the iterator with the return value of `insert()` or `erase()`.

10.2 Passing Vectors by Value

Vectors should almost always be passed to functions by reference (& or `const &`) to avoid copying all elements.

```
void print(vector<int> vec); // BAD: copies entire vector! O(n) overhead
void print(const vector<int>& vec); // GOOD: passes by const reference. O(1) overhead
```

10.3 `size()` vs `capacity()` Confusion

Resizing a vector (`resize(n)`) physically inserts elements and changes its **size**. Reserving memory (`reserve(n)`) only changes the **capacity** without inserting elements.

```
vector<int> v;
v.reserve(10);
cout << v[0]; // BUG: Size is still 0! Accessing v[0] is undefined behavior.
```

11. Quick Revision Summary

- **Vector** = Heap-allocated dynamic array, $O(1)$ random access, automatic growth.
- **Growth Factor** = Usually doubles (2x) or scales by 1.5x capacity when filled.
- **Amortized Push** = $O(1)$ because reallocations happen exponentially less frequently.
- **Key Methods**: `push_back` (add end), `pop_back` (remove end), `reserve` (allocate memory), `resize` (change size).
- **Core Pitfall**: Iterator invalidation on structural changes, and copying vectors by value.